TrOOPL standard library
Version v0.1
For TrOOPL v0.2

The standard library is an environment, providing extensions on top of the default environment.

Since some of the methods of the default environment are modified, descriptions for the whole interface are provided, even when there is no change from default TrOOPL environment.

When methods describe the type of the input they expect, behavior on providing an object that is not of the said type as the argument is unspecified (But in practice will usually result in a crash).

Contents

1. Types 7
   1.1. Boolean 7
      1.1.1. `then-else A B` 7
      1.1.2. `and Other` 7
      1.1.3. `and-then Block` 7
      1.1.4. `or Other` 7
      1.1.5. `or-else Block` 7
      1.1.6. `xor Other` 7
      1.1.7. `-> Other` 7
      1.1.8. `not` 7
      1.1.9. `then Then-Block` 7
         1.1.9.1. `else Else-Block` 8
         1.1.9.2. `else-if Condition-Block Else-If-Block` 8
         1.1.9.3. `end` 8
      1.1.10. `= Other` 8
      1.1.11. `<> Other` 8
      1.1.12. `> Other` 8
      1.1.13. `< Other` 8
      1.1.14. `>= Other` 8
      1.1.15. `<= Other` 8
      1.1.16. `to-string` 8
   1.2. Number 8
      1.2.1. `+ Other` 8
      1.2.2. `- Other` 8
      1.2.3. `-` 8
      1.2.4. `* Other` 8
      1.2.5. `/ Other` 8
      1.2.6. `mod Other` 9
      1.2.7. `rem Other` 9
      1.2.8. `^ Other` 9
      1.2.9. `// Other` 9
      1.2.10. `min Other` 9
      1.2.11. `max Other` 9
      1.2.12. `= Other` 9
      1.2.13. `<> Other` 9
      1.2.14. `> Other` 9
      1.2.15. `< Other` 9
      1.2.16. `>= Other` 9
      1.2.17. `<= Other` 9
      1.2.18. `floor` 9
      1.2.19. `ceil` 10
      1.2.20. `trunc` 10
      1.2.21. `abs` 10
      1.2.22. `inv` 10
      1.2.23. `sqrt` 10
      1.2.24. `to-string` 10
   1.3. String 10

1.3.1. to-number 10
1.3.2. to-bytes 10
1.3.3. to-string 10
1.4. List 11
1.4.1. at Index 11
1.4.2. size 11
1.4.3. map Function 11
1.4.4. zip Other 11
1.4.5. + Other 11
1.4.6. sub Start End 11
1.4.7. find-sub-index Sublist 11
1.4.8. reverse 11
1.4.9. sort 11
1.4.10. bisect Predicate 12
1.4.11. = Other 12
1.4.12. <> Other 12
1.4.13. < Other 12
1.4.14. > Other 12
1.4.15. <= Other 12
1.4.16. >= Other 12
1.4.17. copy 12
1.4.18. to-seq 12
1.4.19. to-string 12
1.5. Array 12
1.5.1. set Index Value 13
1.5.2. push Value 13
1.5.3. pop 13
1.5.4. clear 13
1.5.5. insert Index Value 13
1.5.6. remove Index 13
1.5.7. append Other 13
1.5.8. reverse-in-place 13
1.5.9. sort-in-place 13
1.6. Sequence 13
1.6.1. run 14
1.6.1.1. cons? 14
1.6.1.2. get 14
1.6.1.3. next 14
1.6.2. map Function 14
1.6.3. filter Predicate 14
1.6.4. zip Other 14
1.6.5. + Other 14
1.6.6. take Number 14
1.6.7. drop Number 14
1.6.8. take-while Predicate 14
1.6.9. drop-while Predicate 15
1.6.10. find-index Predicate 15
1.6.11. intersperse Value 15
1.6.12. left-fold Default Function 15

1.6.13. right-fold Default Function 15
1.6.14. left-scan Default Function 15
1.6.15. flatten 15
1.6.16. for-each Function 15
1.6.17. to-string 15
1.6.18. to-list 16
1.6.19. to-array 16
1.7. Error 16
1.7.1. ok? 16
1.7.2. message 16
1.7.3. crash 16
1.8. Pair 16
1.8.1. a 16
1.8.2. b 16
2. Methods 17
2.1. io 17
2.1.1. print String 17
2.1.2. raw-print String 17
2.1.3. write Bytes 17
2.1.4. read-line 17
2.1.5. read N-Bytes 17
2.1.6. flush 17
2.1.7. error 17
2.1.7.1. print String 17
2.1.7.2. raw-print String 18
2.1.7.3. write Bytes 18
2.1.7.4. flush 18
2.2. fs 18
2.2.1. read Filename 18
2.2.2. write Filename String 18
2.2.3. open-read Filename 18
2.2.3.1. read N-Bytes 18
2.2.3.2. close 18
2.2.4. open-write Filename 18
2.2.4.1. write Bytes 19
2.2.4.2. flush 19
2.2.4.3. close 19
2.2.5. metadata Filename 19
2.2.5.1. exists? 19
2.2.5.2. file? 19
2.2.5.3. dir? 19
2.2.5.4. other? 19
2.2.5.5. accessed 19
2.2.5.6. modified 19
2.2.5.7. size 20
2.2.6. real-path Filename 20
2.2.7. rename Filename Filename' 20
2.2.8. make-dir Dirname 20
2.2.9. remove Filename 20

2.2.10. remove-dir Dirname 20
2.2.11. list-dir Dirname 20
2.2.12. working-dir 20
2.2.13. working-dir Directory 20
2.3. system 20
2.3.1. args 21
2.3.2. run Command 21
2.3.3. run Command Input 21
2.3.3.1. stdout 21
2.3.3.2. stderr 21
2.3.3.3. exit-code 21
2.3.4. time 21
2.3.5. exit Exit-Code 21
2.4. utf-8 21
2.4.1. to-bytes String 21
2.4.2. from-bytes Bytes 21
2.5. env 21
2.5.1. run Env Block 21
2.5.2. bind Env Object 22
2.6. eval String 22
2.7. box Value 22
2.7.1. set Value' 22
2.7.2. get 22
2.8. array 22
2.9. true 22
2.10. false 22
2.11. core 22
2.11.1. extend 22
2.11.1.1. boolean Minimal-Boolean 22
2.11.1.2. number Minimal-Number 22
2.11.1.3. string Minimal-String 22
2.11.1.4. list Minimal-List 22
2.11.1.5. array Minimal-Array 23
2.11.1.6. seq Minimal-Seq 23
2.11.1.7. error Minimal-Error 23
2.11.2. tag Number 23
2.11.3. from-seq Seq 23
2.12. filename 23
2.13. crash Message 23
2.14. import Filename 23
2.15. import Module-Env Filename 23
2.16. import-env Filename 23
2.17. import-env Module-Env Filename 23
2.18. raw-import Filename 23
2.19. raw-import Module-Env Filename 23
2.20. list 24
2.20.1. <> 24
2.20.2. < Value 24
2.20.2.1. , Value' 24

2.20.2.2. > 24
2.20.3. range Start End 24
2.20.4. repeat Value Size 24
2.21. pair A B 24
2.22. seq 24
2.22.1. range Start 24
2.22.2. repeat Value 24
2.22.3. iterate First Function 24
2.22.4. single Value 24
2.23. while Condition-Block Loop-Block 24

1. Types

This section describes the interfaces of types used throughout the standard library.

The types describe the values **produced** by the standard library, not consumed. For consumption, the standard library works for any object satisfying the default TrOOPL interfaces — `then-else A B` for booleans, the default number interface for numbers (Numeric tag is always not enough), `at Index` and `size` for strings and lists.

1.1. Boolean

Booleans represent two values - true or false, represented by behavior of the `then-else A B` method, from which the rest of the methods of a boolean are derived. It has the following methods (The boolean the methods are attached to is references as `Self`).

1.1.1. `then-else A B`

Returns `A` if the boolean represents a true value, `B` otherwise.

1.1.2. `and Other`

Returns a new boolean representing the conjunction of `Self` and `Other`.

Specifically, the result is `Self then-else Other Self`.

1.1.3. `and-then Block`

Returns a new boolean representing the conjunction of `Self` and result of block `Block`. `Block` is only evaluated if `Self` is true.

Specifically, the result is `Self then-else Block [Self] run`.

1.1.4. `or Other`

Returns a new boolean representing the disjunction of `Self` and `Other`.

Specifically, the result is `Self then-else Self Other`.

1.1.5. `or-else Block`

Returns a new boolean representing the disjunction of `Self` and result of block `Block`. `Block` is only evaluated if `Self` is false.

Specifically, the result is `Self then-else [Self] Block run`.

1.1.6. `xor Other`

Returns a new boolean representing the exclusive OR of `Self` and `Other`.

Specifically, the result is `Self then-else [Other then-else [false] [Self] run] [Other] run`.

1.1.7. `-> Other`

Returns a new boolean representing "`Self` implies `Other`".

Specifically, the result is `Self then-else [Other] [true] run`.

1.1.8. `not`

Returns a new boolean representing negation of `Self`.

Specifically, the result is `Self then-else [false] [true] run`

1.1.9. `then Then-Block`

Starts a [`then`]-[`else-if`]-[`else`/`end`] chain. Returns an object with following methods.

1.1.9.1. `else Else-Block`

Evaluates `Self then-else Then-Block Else-Block run`.

1.1.9.2. `else-if Condition-Block Else-If-Block`

Creates a new [then]-[else-if]-[else/end] chain object equivalent to `Self then Then-Block` if `Self` is true, and `Condition-Block run then Else-If-Block` otherwise.

1.1.9.3. `end`

Equivalent to calling `else [{}]` on the [then]-[else-if]-[else/end] chain object.

1.1.10. `= Other`

Equivalent to `Self xor Other not`.

1.1.11. `<> Other`

Equivalent to `Self xor Other`.

1.1.12. `> Other`

Equivalent to `Self -> Other not`.

1.1.13. `< Other`

Equivalent to `Other -> Self not`.

1.1.14. `>= Other`

Equivalent to `Other -> Self`.

1.1.15. `<= Other`

Equivalent to `Self -> Other`.

1.1.16. `to-string`

Returns string representation of the boolean.

Specifically, the result is `Self then-else "true" "false"`

1.2. Number

Numbers represent real numeric values, with the actual representation stored in the object's numeric tag. In addition to the numeric tag, numbers produced by the standard library have the following methods (The number to which the methods are attached to is referred to as `Self`).

1.2.1. `+ Other`

Returns a new number representing `Self` + `Other`.

1.2.2. `- Other`

Returns a new number representing `Self` − `Other`.

1.2.3. `-`

Returns a new number representing −`Self`.

1.2.4. `* Other`

Returns a new number representing `Self` · `Other`.

1.2.5. `/ Other`

Returns a new number representing $\frac{\texttt{Self}}{\texttt{Other}}$.

Behavior on `Other` $= 0$ is in accordance with IEEE 754 if infinities and/or NaNs are supported, otherwise unspecified.

1.2.6. `mod Other`

Returns a new number representing `Self` mod `Other` (Remainder of Euclidean division).

Behavior on `Other` $= 0$ is unspecified.

1.2.7. `rem Other`

Returns a new number representing `Self` $\%$ `Other` (Remainder of truncating division).

This value (`Result`) is such that `Self / Other trunc * Other + Result = Self`.

Behavior on `Other` $= 0$ is unspecified.

1.2.8. `^ Other`

Returns a new number representing $\texttt{Self}^{\texttt{Other}}$.

Raising negative numbers to integer powers works without failures. If infinities and/or NaNs are supported, behavior is according to IEEE 754. Otherwise, behavior for raising negatives to non-integer powers is unspecified.

1.2.9. `// Other`

Returns a new number representing $\left\lfloor \frac{\texttt{Self}}{\texttt{Other}} \right\rfloor$.

Behavior on `Other` $= 0$ is in accordance with IEEE 754 if infinities and/or NaNs are supported, otherwise unspecified.

1.2.10. `min Other`

Returns a new number representing $\min(\texttt{Self}, \texttt{Other})$.

1.2.11. `max Other`

Returns a new number representing $\max(\texttt{Self}, \texttt{Other})$.

1.2.12. `= Other`

Returns a new boolean representing comparison `Self` $=$ `Other`.

1.2.13. `<> Other`

Returns a new boolean representing comparison `Self` $\neq$ `Other`.

1.2.14. `> Other`

Returns a new boolean representing comparison `Self` $>$ `Other`.

1.2.15. `< Other`

Returns a new boolean representing comparison `Self` $<$ `Other`.

1.2.16. `>= Other`

Returns a new boolean representing comparison `Self` $\geq$ `Other`.

1.2.17. `<= Other`

Returns a new boolean representing comparison `Self` $\leq$ `Other`.

1.2.18. `floor`

Returns a new number representing $\lfloor \texttt{Self} \rfloor$ (Round number towards negative infinity).

1.2.19. `ceil`

Returns a new number representing $\lceil$`Self`$\rceil$ (Round number towards positive infinity).

1.2.20. `trunc`

Returns a new number representing $\mathrm{trunc}($`Self`$)$ (Round number towards zero).

1.2.21. `abs`

Returns a new number representing $|$`Self`$|$.

1.2.22. `inv`

Returns a new number representing $\frac{1}{\texttt{Self}}$.

1.2.23. `sqrt`

Returns a new number representing $\sqrt{\texttt{Self}}$.

For negatives numbers, returns NaN if supported, otherwise behavior is unspecified.

1.2.24. `to-string`

Returns a new string containing representation of `Self`. Precision is unspecified.

Produces a valid TrOOPL numeric literal. If the number is not representable as a TrOOPL numeric literal, the result is unspecified.

1.3. String

Strings represent sequences of Unicode codepoints, with support for non-Unicode bytes, which are used verbatim instead of being translated to UTF-8 during encoding. During decoding, if invalid UTF-8 bytes are encountered, they are stored in the string as a non-Unicode byte. This is done to support lossless conversion between strings and byte lists.

Valid Unicode code points are represented as nonnegative integers, while a non-Unicode byte B is encoded as $-B$.

Strings produced by standard library have all of the methods from lists where elements are the code points and non-Unicode bytes, and methods produce strings instead of lists. Methods that differ or are not present in lists are described below (The object the methods are attached to is referred to as `Self`).

1.3.1. `to-number`

Returns a recoverable error, with message “parse error”.

Parses the string as a TrOOPL number and returns the result.

Does not parse NaNs or infinities, and doesn’t take negative zeroes into consideration, even if all of them are supported.

1.3.2. `to-bytes`

Equivalent to `utf-8 to-bytes Self`.

1.3.3. `to-string`

Returns the string with each ‘`"`’ escaped according to TrOOPL string literal syntax, and wrapped between ‘`"`’s.

This does not treat carriage returns specially, despite string literals in TrOOPL ignoring them, meaning this will result in lossy conversion if the string contains carriage return characters.

1.4. List

Lists represent sized, indexable sequences of arbitrary elements.

Methods of lists produces by the standard library are described below (The object to which the methods are attached is referred to as `Self`).

1.4.1. `at Index`

Returns the element at index `Index` (number, integer). Lists are 0-indexed.

Crashes on out of bounds access.

1.4.2. `size`

Returns the size of the list as a number.

1.4.3. `map Function`

Returns a new list where each item `X` of self has been replaced with `Function run X`.

This results in a lazy list that delegates indexing to `Self` on each call to `at Index` and `size`. This also doesn't cache the results of `Function run X`, so it will be recalculated on every access. To avoid that, after using this method, call `copy`.

1.4.4. `zip Other`

Returns a new list where each element is a pair of corresponding elements from lists `Self` and `Other`. This results in a lazy list that delegates indexing to `Self` and `Other` on each call to `at Index` and `size`. To avoid that, after using this method, call `copy`.

1.4.5. `+ Other`

Returns a new list representing concatenation of lists `Self` and `Other`. The resulting list has the size of the smaller of two lists.

This results in a lazy list that delegates indexing to `Self` and `Other` on each call to `at Index` and `size`. To avoid that, after using this method, call `copy`.

1.4.6. `sub Start End`

Returns a new list that represents a sublist of `Self`, starting at index `Start` (number, integer, inclusive), and ending at `End` (number, integer, exclusive). No bound checks or input validation is done, so this might result in an invalid list.

This results in a lazy list that delegates indexing to `Self` on each call to `at Index`. To avoid that, after using this method, call `copy`.

1.4.7. `find-sub-index Sublist`

Returns the index of the first sublist equal to `Sublist` (Via `= Other`). Returns (`Self size`) − (`Sublist size`)`+1` if not found. Empty `Sublist` is always found, at index 0.

1.4.8. `reverse`

Returns a new list that represents `Self` reversed.

This results in a lazy list that delegates indexing to `Self` on each call to `at Index` and `size`. To avoid that, after using this method, call `copy`.

1.4.9. `sort`

Returns a new list that represents `Self` sorted. This method allocates a new underlying list, not merely a proxy object.

Sorting is done by using `<= Other` method on the elements. The sort is stable.

1.4.10. `bisect Predicate`

Returns index where for element `X`, `Predicate run X` is true, or length of the list if no such element was found, in logarithmic time, assuming the list is ordered in such a was that `Predicate run X` returns false for all elements before that index, and true for all elements after it.

1.4.11. `= Other`

Returns true if sizes of `Self` and `Other` match and for all elements `X` of `Self` and corresponding elements `Y` of `Other`, `X = Y` returns true, otherwise false. Stops iteration on first non-equal pair of elements.

1.4.12. `<> Other`

Negation of `Self = Other`.

1.4.13. `< Other`

Returns true if `Self` is lexicographically less than `Other`, comparing elements of `Self` (`X`) to elements of `Other` (`Y`) using `X = Y`, and `X < Y`, otherwise false. Stops iteration on first element where `X = Y` returns false, then decides result of the comparison based on result of `X < Y`. A proper prefix of a list is considered to be the lesser one.

1.4.14. `> Other`

Returns true if `Self` is lexicographically less than `Other`, comparing elements of `Self` (`X`) to elements of `Other` (`Y`) using `X = Y`, and `X > Y`, otherwise false. Stops iteration on first element where `X = Y` returns false, then decides result of the comparison based on result of `X > Y`. A proper prefix of a list is considered to be the lesser one.

1.4.15. `<= Other`

Negation of `Self > Other`.

1.4.16. `>= Other`

Negation of `Self < Other`.

1.4.17. `copy`

Returns a new list with same elements as in `Self`, reconstructing it with `from-seq` to ensure $O(1)$ `at Index` and `size`.

1.4.18. `to-seq`

Returns a new sequence with elements from the list.

The sequence will be lazy and delegate to the list. If that's a problem, create a copy of the list before converting it to a sequence.

1.4.19. `to-string`

Returns a new string, which is "`list <>`" if the list is empty, and otherwise the results of `to-string` of each element joined with "` , `" in between, wrapped between "`list < `" and "` >`"

1.5. Array

An array represents a mutable growable ordered collection of elements. Arrays have all of the methods of a list. Those methods behave exactly the same as methods of a list, except that `copy` returns an array instead of a list (Other list-returning methods still return lists). Methods of arrays that are not methods of lists or array-specific changes are described below.

1.5.1. `set Index Value`

Sets the value of element at index `Index` (number, integer) to `Value`. Crashes if the index is out of bounds.

Returns the array itself.

1.5.2. `push Value`

Add `Value` as the last element of the array.

Returns the array itself.

1.5.3. `pop`

Removes the last element of the array and returns it.

Crashes if the array is empty.

1.5.4. `clear`

Removes all elements from the array.

Returns the array itself.

1.5.5. `insert Index Value`

Inserts `Value` at index `Index` (number, integer), shifting values to the right of it. Equivalent to `push Value` if `Index` is equal to the size of the array.

Crashes on out of bounds insertion.

Returns the array itself.

1.5.6. `remove Index`

Removes value at index `Index` (number, interger), shifting values on the right of it 1 element to the left.

Crashes on out of bounds removal.

Returns the removed element.

1.5.7. `append Other`

Appends all of the elements of the list `Other` to the end of the array.

Returns the array itself.

1.5.8. `reverse-in-place`

Reverses the array in-place.

Returns the array itself.

1.5.9. `sort-in-place`

Sorts the array in-place.

Returns the array itself.

1.6. Sequence

A sequence represents a lazy, potentially infinite sequence of values organized in a way similar to singly-linked lists. Elements are not computed until iteration. While generally it is not guaranteed whether querying elements from the same sequence multiple times results in caching, recalculating said elements, or even side effects, following methods produce sequences that recalculate their

results every time, and for one full consumption of the resulting sequence consume the input sequences from which the result was constructed at most once.

Sequences have the following methods (The object they are attached to is referred to as `Self`).

1.6.1. `run`

Computes and returns the current sequence node. The sequence node has the following methods.

1.6.1.1. `cons?`

Returns true if this node is a cons node, false otherwise.

1.6.1.2. `get`

Only exists if the sequence node is a cons node.

Returns the value stored in this sequence node.

1.6.1.3. `next`

Only exists if the sequence node is a cons node.

Returns the next sequence item.

1.6.2. `map Function`

Constructs a new lazy sequence where for each element `X` in `Self` the element would be `Function run X`.

1.6.3. `filter Predicate`

Constructs a new lazy sequence where for each element `X` in `Self` the element is only included if `Predicate run X` is true.

1.6.4. `zip Other`

Constructs a new lazy sequence where elements are ordered pairs of the corresponding elements in `Self` and `Other`. The resulting sequence ends when either `Self` or `Other` ends.

Check order is `Self` then `Other`, so if while computing the next element it turns out that `Self` reached the end, no attempt to compute the next element of `Other` will be made.

1.6.5. `+ Other`

Returns a new lazy sequence consisting of all items in `Self` followed by items in `Other`.

1.6.6. `take Number`

Returns a new lazy sequence with same elements as in `Self`, but limited to at most `Number` of elements.

Negative `Number` results in no elements being taken.

1.6.7. `drop Number`

Returns a new lazy sequence with same elements as in `Self`, but with first `Number` of elements dropped. If the sequence had less items that `Number`, the resulting sequence is empty.

Negative `Number` results in no elements being dropped.

1.6.8. `take-while Predicate`

Returns a new lazy sequence with same elements as in `Self`, but only taking elements until `Predicate run X` (Where `X` is the element being tested) returns false.

1.6.9. `drop-while Predicate`

Returns a new lazy sequence with same elements as in `Self`, but dropping elements until `Predicate run X` (Where `X` is the element being tested) returns false.

1.6.10. `find-index Predicate`

Returns the 0-based index of the first element `X` for which `Predicate run X` is true as a number, or size of the sequence if no such element was found.

1.6.11. `intersperse Value`

Puts `Value` in between each pair of consecutive elements of the sequence. Does nothing to sequences of size 0 and 1.

1.6.12. `left-fold Default Function`

Performs a left fold over the sequence, repeatedly calculating `Function run Accumulator Value`, where `Accumulator` is the result of the previous call to `Function run Accumulator Value` (`Default` at the start), and `Value` is the current sequence element.

The result is the result of the last call of `Function run Accumulator Value`.

1.6.13. `right-fold Default Function`

Performs a right fold over the sequence, repeatedly calculating `Function run Value Accumulator-Block`, where `Accumulator-Block` is a block that calculates the result of the next call to `Function run Value Accumulator-Block` (`Default` when sequence has reached its end), and `Value` is the current sequence element.

This method might result in a stack overflow for sequences that are too long as it is not tail recursive.

The iteration only progresses if `run` is called on `Accumulator-Block`, allowing the fold to short-circuit.

The result is the result of the first call of `Function run Value Accumulator-Block`.

1.6.14. `left-scan Default Function`

Performs a left scan over the sequence, repeatedly calculating `Function run Accumulator Value`, where `Accumulator` is the result of the previous call to `Function run Accumulator Value` (`Default` at the start), and `Value` is the current sequence element.

The result is a lazy sequence where first element is `Default` and each element after that is the result of the corresponding `Function run Accumulator Value` call. This means that the resulting sequence will have 1 more element than `Self`.

1.6.15. `flatten`

Returns a new lazy sequence removing one layer of nesting of the sequence of sequences, transforming it into one big sequence.

1.6.16. `for-each Function`

For each element `X`, in order, evaluates `Function run X`.

Returns `{}`.

1.6.17. `to-string`

Converts this sequence to a new string.

1.6.18. `to-list`

Converts this sequence to a new list.

1.6.19. `to-array`

Converts this sequence to a new array.

1.7. Error

Recoverable errors represent values that might fail. When they represent successful results, they have all of the methods that the successful result should have, in addition to `ok?` and `crash`. Otherwise, they only have `ok?`, `message`, and `crash`.

1.7.1. `ok?`

Returns true if the value represents success, false otherwise.

1.7.2. `message`

Only exists if the value represents an error.

Returns the string containing description of the error.

1.7.3. `crash`

Crashes the program with the same message as returned `message` if the value represents an error, otherwise returns the recoverable error object itself.

1.8. Pair

Pairs represent ordered pairs. They have the following methods.

1.8.1. `a`

Returns the first item of the pair.

1.8.2. `b`

Returns the second item of the pair.

2. Methods

This section describes the methods the standard library provides.

2.1. `io`

Returns a stateless object with the following set of methods.

2.1.1. `print String`

Prints string `String` to the standard output, followed by a newline, and flushes the output buffer.

Returns a recoverable error.

Return value on success — `io` object itself.

2.1.2. `raw-print String`

Prints string `String` to the standard output, without flushing or appending a newline.

Returns a recoverable error.

Return value on success — `io` object itself.

2.1.3. `write Bytes`

Writes bytes in the list `Bytes` to the standard output.

Returns a recoverable error.

Return value on success — `io` object itself.

2.1.4. `read-line`

Returns a recoverable error.

Reads from standard input until a newline or end of file (Consuming the newline), and returns the result as a string, with the newline stripped.

Returns an error if it was called when end of the input stream was already reached. The line is returned normally if there were some characters to consume before encountering end of stream.

Return value on error has an extra method — `eof?`, which returns true if the error happened due to being called when end of the input stream was already reached, false otherwise.

2.1.5. `read N-Bytes`

Returns a recoverable error.

Reads at most `N-Bytes` (number, must be a positive integer) bytes from standard input and returns them in a list.

If the returned list is empty, that means end of the input stream was reached.

2.1.6. `flush`

Flushes the standard output buffer.

Returns a recoverable error.

Return value on success — `io` object itself.

2.1.7. `error`

Returns a stateless object with methods for printing to standard error.

2.1.7.1. `print String`

Like `io print String`, but prints to standard error, and returns `io error` object on success.

2.1.7.2. `raw-print String`

Like `io raw-print String`, but prints to standard error, and returns `io error` object on success.

2.1.7.3. `write Bytes`

Like `io write Bytes`, but writes to standard error, and returns `io error` object on success.

2.1.7.4. `flush`

Like `io flush`, but flushes standard error, and returns the `io error` object on success.

2.2. `fs`

Returns a stateless object with the following set of methods.

2.2.1. `read Filename`

Returns a recoverable error.

Reads all of the contents of file with filename `Filename` (string) and returns the result as a string.

Unlike in languages like C, this performs no line ending normalization — all of the bytes are preserved.

2.2.2. `write Filename String`

Writes string `String` into a file with filename `Filename` (string).

Returns a recoverable error.

Return value on success — `fs` object itself.

Unlike in languages like C, this performs no line ending normalization — all of the bytes are preserved.

2.2.3. `open-read Filename`

Returns a recoverable error.

Opens file with filename `Filename` (string) for reading and returns the read handle. The methods of the read handle are described below.

2.2.3.1. `read N-Bytes`

Returns a recoverable error.

Reads at most `N-Bytes` (number, must be a positive integer) bytes from the file and returns them in a list.

If the returned list is empty, that means end of file was reached.

This method crashes if the file was already closed.

2.2.3.2. `close`

Closes the file associated with the handle.

Returns a recoverable error.

Return value on success — an empty object.

2.2.4. `open-write Filename`

Returns a recoverable error.

Opens file with filename `Filename` (string) for writing and returns the write handle. The methods of the write handle are described below.

2.2.4.1. `write Bytes`

Writes bytes in the list `Bytes` to the file.

Returns a recoverable error.

Return value on success — the write handle itself.

This method crashes if the file was already closed.

2.2.4.2. `flush`

Flushes the buffer associated with the file.

Returns a recoverable error.

Return value on success — the write handle itself.

This method crashes if the file was already closed.

2.2.4.3. `close`

Closes the file associated with the handle.

Returns a recoverable error.

Return value on success — an empty object.

2.2.5. `metadata Filename`

Returns a recoverable error.

Returns an object containing metadata for the file with name `Filename`. The behavior is analogous to `stat`, so symlinks are followed. If a symlink points to a file that does not exist, that is treated as if the requested file does not exist.

The objects returned by the accessor methods returned by `metadata Filename` are created when `metadata Filename` is called, so the accessor methods return same instances every time instead of reconstructing them.

The returned object is described below.

2.2.5.1. `exists?`

Returns true if the file exists, false otherwise.

2.2.5.2. `file?`

Returns true if the file exists and is regular file, false otherwise

2.2.5.3. `dir?`

Returns true if the file exists and is a directory, false otherwise

2.2.5.4. `other?`

Returns true if the file exists and is neither a regular file, nor a directory, false otherwise

2.2.5.5. `accessed`

Returns the time when the file was last accessed as number representing the Unix timestamp, in seconds, or 0 if the file doesn't exist.

2.2.5.6. `modified`

Returns the time when the file was last modified as number representing the Unix timestamp, in seconds, or 0 if the file doesn't exist.

2.2.5.7. `size`

Returns the total size of the file in bytes, or 0 if it doesn't exist, as a number.

2.2.6. `real-path Filename`

Returns a recoverable error.

Returns the absolute path to file at path `Filename` (string), without `.`, `..`, and symlinks.

The file must exist.

2.2.7. `rename Filename Filename'`

Renames the file at `Filename` (string) to `Filename'` (string).

Returns a recoverable error.

Return value on success — `fs` object itself.

2.2.8. `make-dir Dirname`

Creates an empty directory named `Dirname` (string).

Returns a recoverable error.

Return value on success — `fs` object itself.

2.2.9. `remove Filename`

Removes a non-directory file named `Filename` (string).

Returns a recoverable error.

Return value on success — `fs` object itself.

2.2.10. `remove-dir Dirname`

Removes an empty directory named `Dirname` (string).

Returns a recoverable error.

Return value on success — `fs` object itself.

2.2.11. `list-dir Dirname`

Returns a recoverable error.

Returns the list of all files in the directory named `Dirname` (string), as a list of strings. Each string contains only the name of the file listed, not absolute or relative path. The list excludes `.` and `..`.

2.2.12. `working-dir`

Returns a recoverable error.

Returns the path to the current working directory as a string.

2.2.13. `working-dir Directory`

Sets the current working directory to `Directory` (string).

Returns a recoverable error.

Return value on success — `fs` object itself.

2.3. `system`

Returns a stateless object with the following set of methods.

2.3.1. `args`

Returns the command line arguments passed to the program as a list of strings. The list is guaranteed to have at least one element and first one corresponds to the name of the entry point file.

2.3.2. `run Command`

Like `system run Command Input`, but with empty standard input. Does not construct the empty string for that.

2.3.3. `run Command Input`

Runs the command described by the list of strings `Command` (First element is the executable, the rest are arguments. The executable is searched in `$PATH` if applicable), with its standard input being supplied from the string `Input`.

Returns a recoverable error.

Returns an object describing the result of the command. The object only has accessor methods, which return objects created ahead of time instead of being recreated with every call to the accessors.

The returned object is described below.

2.3.3.1. `stdout`

Returns the standard output of the command as a string.

2.3.3.2. `stderr`

Returns the standard error of the command as a string.

2.3.3.3. `exit-code`

Returns the exit code of the command as a number.

In case the command was terminated by signal X, the value is $-X$.

2.3.4. `time`

Returns the current time as a number representing the Unix timestamp, in seconds.

2.3.5. `exit Exit-Code`

Terminates the program with exit code `Exit-Code` (number, must be an integer).

The exact requirements for `Exit-Code` and how it is used are platform-dependent.

2.4. `utf-8`

Returns a stateless object with the following set of methods.

2.4.1. `to-bytes String`

Encodes the string `String` in UTF-8 and returns the result as a list of numbers, with each element representing a byte.

2.4.2. `from-bytes Bytes`

Decodes the list of numbers `Bytes` as a list of UTF-8 bytes and returns the result as a string.

2.5. `env`

Returns a stateless object with the following set of methods.

2.5.1. `run Env Block`

Evaluates `Block run` under environment `Env`.

2.5.2. `bind Env Object`

Returns an object with same interface as `Object`, except all of the methods run under environment `Env`.

The behavior is analogous to wrapping each method definition in `env run Env [...]`, but doesn't require manually wrapping each individual method definition.

2.6. `eval String`

Evaluates string `String` as TrOOPL source code under a fresh lexical scope, with current environment.

2.7. `box Value`

Constructs a mutable box that holds `Value` by default. The box has the following set of methods.

2.7.1. `set Value'`

Replaces the contents of the box to be `Value'`.

Returns the box itself.

2.7.2. `get`

Returns the value currently contained in the box.

2.8. `array`

Returns a new standard array.

2.9. `true`

Has the exact behavior of `core extend boolean {then-else A B (A)}`

2.10. `false`

Has the exact behavior of `core extend boolean {then-else A B (B)}`

2.11. `core`

Returns a stateless object with the following set of methods.

2.11.1. `extend`

Returns a stateless object with the following set of methods.

2.11.1.1. `boolean Minimal-Boolean`

Extends the default TrOOPL boolean `Minimal-Boolean` with methods of a standard boolean.

Retains the extra methods `Minimal-Boolean` might have.

2.11.1.2. `number Minimal-Number`

Extends the default TrOOPL number `Minimal-Number` with methods of a standard number.

Retains the extra methods `Minimal-Number` might have.

2.11.1.3. `string Minimal-String`

Extends the default TrOOPL string `Minimal-String` with methods of a standard string.

Retains the extra methods `Minimal-String` might have.

This method does not delegate to `core extend list Minimal-List`.

2.11.1.4. `list Minimal-List`

Extends the default TrOOPL list `Minimal-List` with methods of a standard list.

Retains the extra methods `Minimal-List` might have.

2.11.1.5. `array Minimal-Array`

Extends the default TrOOPL array `Minimal-Array` with methods of a standard array.

Retains the extra methods `Minimal-Array` might have.

This method delegates to `core extend list Minimal-List` to add the list methods.

2.11.1.6. `seq Minimal-Seq`

Extends the default TrOOPL sequence `Minimal-Seq` with methods of a standard sequence.

Retains the extra methods `Minimal-Seq` might have.

2.11.1.7. `error Minimal-Error`

Extends the default TrOOPL recoverable error `Minimal-Error` with `crash` from a standard recoverable error.

Retains the extra methods `Minimal-Error` might have.

2.11.2. `tag Number`

Returns an object with no methods, containing the same numeric tag as number `Number`.

2.11.3. `from-seq Seq`

Returns a default TrOOPL list constructed from the sequence `Seq`.

This method does not call `core extend list`.

2.12. `filename`

Returns the filename of the file currently being executed.

Called by the interpreter when reporting an error during a crash.

2.13. `crash Message`

Crashes the program with message string `Message`.

2.14. `import Filename`

Equivalent to `import () Filename`.

2.15. `import Module-Env Filename`

Reads file `Filename` (string) and evaluates it as TrOOPL source under environment `Module-Env`, then returns its result bound to that environment.

2.16. `import-env Filename`

Equivalent to `import-env () Filename`.

2.17. `import-env Module-Env Filename`

Reads file `Filename` (string) and evaluates it as TrOOPL source under environment `Module-Env`, then returns its result (`Result`), bound to `{Module-Env Result}`.

2.18. `raw-import Filename`

Equivalent to `raw-import () Filename`.

2.19. `raw-import Module-Env Filename`

Reads file `Filename` (string) and evaluates it as TrOOPL source under environment `Module-Env`, then returns its result.

2.20. `list`

Returns a stateless object with the following set of methods.

2.20.1. `<>`

Returns a new empty list.

2.20.2. `< Value`

Returns a list constructor with value `Value` commited as the first element.

The methods of a list constructor are described below.

2.20.2.1. `, Value'`

Returns a new list constructor with `Value'` commited as the last element of the list in addition to whatever the constructor had commited before that.

2.20.2.2. `>`

Returns a new list consisting of all of the elements commited to the list constructor.

2.20.3. `range Start End`

Returns a new list consisting of numbers from `Start` (inclusive) to `End` (exclusive), with step of 1.

2.20.4. `repeat Value Size`

Returns a new list of size `Size` where each element is `Value`.

2.21. `pair A B`

Returns a new ordered pair of `A` and `B`.

2.22. `seq`

Returns a stateless object with the following set of methods.

2.22.1. `range Start`

Returns a new infinite sequence of numbers starting from `Start`, where each element is 1 bigger than the previous one.

2.22.2. `repeat Value`

Returns a new lazy infinite sequence where every element is `Value`.

2.22.3. `iterate First Function`

Returns a new lazy infinite sequence where first element is `First`, and each subsequent element is computed by calling `Function run X` where `X` is the last element of the sequence so far.

2.22.4. `single Value`

Returns a new sequence that consist of a single element `Value`.

2.23. `while Condition-Block Loop-Block`

Evaluates `Condition-Block run`, and if its boolean result is true, evaluates `Loop-Block run` and repeats the cycle. If `Condition-Block run` returns false, iteration stops and the method returns `{}`.